CMSC131: Arrays and Indexing

Chapter 8

By Naren Ayinala

May 2026

Contents

1 Why Do We Need Arrays? **3**

2 Declaring, Creating, and Initializing Arrays **3**

2.1 Default Element Values 3

3 Indexing: How to Access Individual Elements **4**

3.1 Reading and Writing Are Different 4

3.2 Indices Can Be Expressions 4

4 Bounds: What Happens When an Index Is Invalid? **5**

5 The `length` Field and Looping Through Arrays **5**

6 For-Each Loops: Readable Traversal **6**

7 Common Array Algorithms **6**

7.1 Sum and Average 6

7.2 Finding a Maximum and Its Index 7

7.3 Counting Matches 7

7.4 Linear Search 7

7.5 Reversing an Array in Place 7

8 Arrays Are Objects: References, Aliasing, and Memory **8**

8.1 Passing an Array to a Method 8

9 Printing and Comparing Arrays **9**

9.1 Copying with `clone()` and `arraycopy()` 9

10 Java Arrays Cannot Be Sliced Like Python Lists **9**

11 Arrays of Strings and Other Objects **10**

12 Array Length Is Fixed: Arrays Versus ArrayList **10**

13 Introduction to Two-Dimensional Arrays **11**

14 Worked Example: Analyze a Class's Scores **11**

15 Final Comparison of Common Array Operations **12**

1 Why Do We Need Arrays?

Suppose a program needs to remember the scores of five students. We could declare five separate variables: `score1`, `score2`, and so on. But what if the class contains 100 students? Writing 100 variables and 100 separate statements would be tedious, and processing them together would be awkward. An **array** lets us store a fixed number of values of one type in an indexed collection. We can use a loop to access each value instead of repeating nearly identical code.

```
int score1 = 90, score2 = 82, score3 = 95;
// Versus:
int[] scores = {90, 82, 95};
for (int i = 0; i < scores.length; i++) {
    System.out.println(scores[i]);
}
```

An array is an **object** in Java. Its elements are stored in an ordered sequence, and its length is fixed when the array is created. Every element has the same declared element type. An array of `int` holds integer values; an array of `String` holds String references.

2 Declaring, Creating, and Initializing Arrays

These are three related but different ideas. **Declaring** a variable specifies its type. **Creating** an array allocates an array object. **Initializing** its elements provides starting values.

```
int[] numbers; // Declaration: no array created yet.
numbers = new int[4]; // Create array with four int elements.
numbers[0] = 12; // Assign first element.
numbers[1] = 24;
```

We usually prefer `int[] numbers` over the also-legal `int numbers[]`, because the first form makes the element type more obvious. The number in `new int[4]` is the *length*, not the last index. It creates four positions with indices 0, 1, 2, and 3.

```
int[] values = new int[3];
int[] primes = {2, 3, 5, 7};
String[] names = {"Ada", "Grace", "Alan"};
double[] prices = new double[] {4.5, 9.0, 12.75};
```

The brace initializer `{...}` works as part of a declaration. When assigning a newly created array later, use `new int[] {...}` rather than just braces.

```
int[] a;
a = new int[] {10, 20, 30}; // Legal.
// a = {10, 20, 30}; // Compile-time error.
```

2.1 Default Element Values

A newly created array automatically contains default values. This is different from an uninitialized local primitive variable, which cannot be read before assignment.

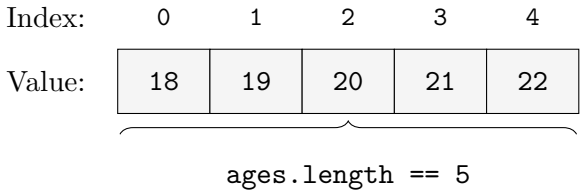
Array element type	Default value
`int`, `long`, etc.	`0`
`double`, `float`	`0.0`
`boolean`	`false`
`char`	`0 (null character)` (null character)
Reference types (e.g. `String`)	`null`

```
int[] counts = new int[3];
System.out.println(counts[0]); // 0
String[] words = new String[2];
System.out.println(words[0]); // null
// words[0].length(); // NullPointerException
```

3 Indexing: How to Access Individual Elements

Java uses **zero-based indexing**: the first element is at index 0. For an array of length n, the valid indices are 0 through $n - 1$. An index is the element's *position*, not its value.

```
int[] ages = {18, 19, 20, 21, 22};
System.out.println(ages[0]); // 18: first element
System.out.println(ages[2]); // 20: third element
System.out.println(ages[4]); // 22: last element
System.out.println(ages.length); // 5
```



3.1 Reading and Writing Are Different

`ages[2]` can appear on the right of an assignment to *read* a value, or on the left to *replace* an element.

```
int[] ages = {18, 19, 20};
int saved = ages[1]; // saved becomes 19
ages[1] = 25; // Change second array element
ages[0] += 2; // First element changes from 18 to 20
ages[2]++; // Third element changes from 20 to 21
// Array is now {20, 25, 21}; saved is still 19.
```

The variable `saved` received a copy of the primitive value. It does not remain linked to that array element.

3.2 Indices Can Be Expressions

Anything evaluating to an appropriate integer index can go inside square brackets. Evaluate the index expression before looking up the element.

```
int[] data = {4, 8, 12, 16, 20};
int i = 1;
System.out.println(data[i + 2]); // data[3] -> 16
System.out.println(data[++i]); // i becomes 2, prints 12
System.out.println(data[i++]); // prints 12, then i becomes 3
System.out.println(data[i]); // 16
```

4 Bounds: What Happens When an Index Is Invalid?

Java checks array bounds at runtime. Accessing a negative index or an index greater than or equal to the length throws an `ArrayIndexOutOfBoundsException`.

```
int[] values = {10, 20, 30};
System.out.println(values[2]); // 30
// System.out.println(values[3]); // Runtime exception
// System.out.println(values[-1]); // Runtime exception
```

A common mistake is confusing the length with the last index. For a length-3 array, the last index is 2, *not* 3. An empty array has length 0 and no valid index.

```
int[] empty = new int[0];
System.out.println(empty.length); // 0
// empty[0] = 5; // Runtime exception
```

5 The `length` Field and Looping Through Arrays

An array exposes a `length` *field*, not a method. Write `arr.length`, not `arr.length()`. Strings are different: their method is `text.length()`.

```
int[] nums = {3, 6, 9};
System.out.println(nums.length); // 3
String text = "Java";
System.out.println(text.length()); // 4
```

A standard array traversal starts at 0 and continues while the index is *less than* the array length.

```
int[] nums = {5, 10, 15, 20};
for (int i = 0; i < nums.length; i++) {
    System.out.println("Index " + i + ": " + nums[i]);
}
```

Iteration	`i`	Condition	Printed value
1	0	`0 < 4` true	5
2	1	`1 < 4` true	10
3	2	`2 < 4` true	15
4	3	`3 < 4` true	20
After update	4	`4 < 4` false	none

Using `i <= nums.length` instead would attempt `nums[4]` and throw an exception. For a reverse traversal, start at `nums.length - 1` and move toward zero.

```
for (int i = nums.length - 1; i >= 0; i--) {
    System.out.print(nums[i] + " "); // 20 15 10 5
}
```

6 For-Each Loops: Readable Traversal

Java's enhanced `for` loop lets us visit each element without writing an index. This is useful when we need the *values* but not their positions.

```
int[] numbers = {4, 7, 10};
for (int number : numbers) {
    System.out.println(number);
}
```

Read the header as "for each `number` in `numbers`." For primitive arrays, `number` receives a copy of each primitive element. Changing that local variable does not change the array.

```
int[] values = {1, 2, 3};
for (int value : values) {
    value *= 10;
}
System.out.println(values[0]); // 1, not 10
```

To update primitive array elements, use an indexed loop:

```
for (int i = 0; i < values.length; i++) {
    values[i] *= 10;
}
// Now {10, 20, 30}.
```

For reference arrays, the for-each variable is a copy of the reference; calling a mutating method through it can still change the referenced object. But reassigning the loop variable does not replace an element in the array.

7 Common Array Algorithms

Once we understand indexing, we can solve many problems with one loop. The key is to choose an initial value, visit every relevant element, and update a running result.

7.1 Sum and Average

```
int[] grades = {80, 90, 100, 70};
int sum = 0;
for (int i = 0; i < grades.length; i++) {
    sum += grades[i];
}
double average = (double) sum / grades.length;
```

```
System.out.println(sum); // 340
System.out.println(average); // 85.0
```

Casting before division avoids integer division. If the array might be empty, check `grades.length > 0` before dividing.

7.2 Finding a Maximum and Its Index

```
int[] scores = {72, 95, 88, 95, 60};
int maxIndex = 0;
for (int i = 1; i < scores.length; i++) {
    if (scores[i] > scores[maxIndex]) {
        maxIndex = i;
    }
}
System.out.println(maxIndex); // 1
System.out.println(scores[maxIndex]); // 95
```

Because we used `>` rather than `>=`, the first occurrence of the maximum wins. This example assumes the array is nonempty.

7.3 Counting Matches

```
int[] data = {2, 5, 2, 8, 2};
int count = 0;
for (int value : data) {
    if (value == 2) {
        count++;
    }
}
System.out.println(count); // 3
```

7.4 Linear Search

A linear search checks elements in order until a match is found. Return the index, or -1 if no match exists.

```
public static int findIndex(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            return i;
        }
    }
    return -1;
}
// findIndex(new int[] {8, 3, 6}, 3) returns 1.
```

7.5 Reversing an Array in Place

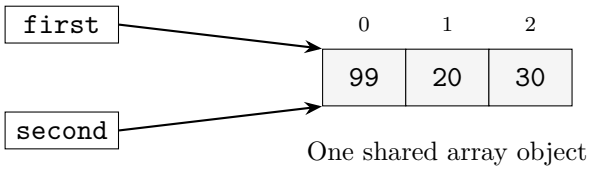
We can swap elements at opposite ends, then move inward. Only half the positions need to be visited.

```
int[] arr = {1, 2, 3, 4, 5};
for (int left = 0; left < arr.length / 2; left++) {
    int right = arr.length - 1 - left;
    int temp = arr[left];
    arr[left] = arr[right];
    arr[right] = temp;
}
// arr is now {5, 4, 3, 2, 1}.
```

8 Arrays Are Objects: References, Aliasing, and Memory

An array variable holds a *reference* to an array object. Assigning one array variable to another copies the reference, *not* the array elements.

```
int[] first = {10, 20, 30};
int[] second = first;
second[0] = 99;
System.out.println(first[0]); // 99
```



One shared array object

To make an independent array, explicitly copy its elements. Java provides convenient copying tools.

```
import java.util.Arrays;
int[] original = {10, 20, 30};
int[] copy = Arrays.copyOf(original, original.length);
copy[0] = 99;
System.out.println(original[0]); // 10
```

8.1 Passing an Array to a Method

Java passes arguments **by value**, including array references. The called method receives a copy of the reference and can modify the same array object. Reassigning the parameter, however, does not change the caller's reference.

```
public static void modify(int[] data) {
    data[0] = 42; // Changes caller's array.
    data = new int[] {7, 8}; // Only local parameter changes.
}
public static void main(String[] args) {
    int[] nums = {1, 2, 3};
    modify(nums);
    System.out.println(nums[0]); // 42
    System.out.println(nums.length); // 3
}
```

The array object generally lives in the heap, while a local reference variable belongs to its method's local execution context (often described using the call stack). This is a conceptual memory model; the JVM may optimize physical storage.

9 Printing and Comparing Arrays

Printing an array variable directly does not display its elements in a friendly format. Use `Arrays.toString()` for a one-dimensional array.

```
import java.util.Arrays;
int[] a = {1, 2, 3};
System.out.println(a); // Type/identity-style text
System.out.println(Arrays.toString(a)); // [1, 2, 3]
```

As with other objects, `==` tests whether two references identify the same array object. Array `equals()` does not compare elements either. Use `Arrays.equals()` for element-wise equality.

```
int[] a = {1, 2, 3};
int[] b = {1, 2, 3};
System.out.println(a == b); // false
System.out.println(a.equals(b)); // false
System.out.println(Arrays.equals(a,b)); // true
```

9.1 Copying with `clone()` and `arraycopy()`

```
int[] a = {4, 5, 6};
int[] b = a.clone(); // Independent one-dimensional int array
int[] c = new int[3];
System.arraycopy(a, 0, c, 0, a.length);
System.out.println(Arrays.toString(c)); // [4, 5, 6]
```

These are *shallow* copies: for arrays containing references to mutable objects, the new array still refers to the same underlying element objects.

10 Java Arrays Cannot Be Sliced Like Python Lists

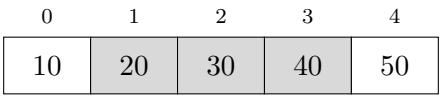
Python allows syntax such as `items[1:4]` to create a slice. **Java arrays do not support colon-based slicing syntax.** A Java array index expression accesses one element, not a range.

```
int[] nums = {10, 20, 30, 40, 50};
// int[] part = nums[1:4]; // INVALID Java syntax.
int element = nums[1]; // 20: one element only.
```

To obtain part of an array, use `Arrays.copyOfRange(array, from, to)`. The starting index is *inclusive* and the ending index is *exclusive*, much like Python slicing bounds.

```
import java.util.Arrays;
int[] nums = {10, 20, 30, 40, 50};
int[] part = Arrays.copyOfRange(nums, 1, 4);
System.out.println(Arrays.toString(part)); // [20, 30, 40]
```

```
part[0] = 999;
System.out.println(nums[1]); // 20: original unchanged
```



indices 1, 2, 3 copied; index 4 excluded

`Arrays.copyOfRange()` creates a new array. It is not a live view into the original. If you only want to *process* part of an array, you may not need a copy at all:

```
for (int i = 1; i < 4; i++) {
    System.out.print(nums[i] + " "); // 20 30 40
}
```

Important difference: Java's `copyOfRange` has specific bounds behavior. A negative start index or a start greater than the array length is invalid; an end index larger than the original length is allowed and fills extra positions with default values. This is different from the forgiving bounds of many Python slices.

11 Arrays of Strings and Other Objects

An array of reference types stores *references* in its elements, not the objects' complete contents directly. The elements may also be `null`.

```
String[] pets = {"cat", "dog", "bird"};
System.out.println(pets[0].length()); // 3
pets[1] = "fish";
System.out.println(pets[1]); // fish
```

String content comparison still uses `equals()`, not `==`:

```
String[] words = {new String("hi"), new String("hi")};
System.out.println(words[0] == words[1]); // false
System.out.println(words[0].equals(words[1])); // true
```

An array of custom objects behaves similarly:

```
Car[] cars = new Car[2];
// cars[0].drive(); // NullPointerException: element is null.
cars[0] = new Car(); // Assuming Car has a no-arg constructor.
```

12 Array Length Is Fixed: Arrays Versus ArrayList

After an array has been created, its `length` cannot change. We can replace its elements, but we cannot append a sixth position to a length-five array. To store a larger collection in an array, create a new array and copy the existing values.

```
int[] old = {1, 2, 3};
int[] bigger = Arrays.copyOf(old, 5);
System.out.println(Arrays.toString(bigger)); // [1, 2, 3, 0, 0]
```

`ArrayList` is a separate class that can grow and shrink dynamically. It uses methods such as `add()` and `get()` instead of bracket indexing. We will study it in detail in the next chapter.

13 Introduction to Two-Dimensional Arrays

A two-dimensional Java array is an *array of arrays*. It is useful for tables, grids, and matrices. Two indices are used: one for the row and one for the column.

```
int[][] grid = {
    {1, 2, 3},
    {4, 5, 6}
};
System.out.println(grid[0][0]); // 1
System.out.println(grid[1][2]); // 6
System.out.println(grid.length); // 2 rows
System.out.println(grid[0].length); // 3 columns in row 0
```

	col 0	col 1	col 2
row 0	1	2	3
row 1	4	5	6

Rows do not have to be the same length. This is called a *jagged array*.

```
int[][] jagged = {{1}, {2, 3}, {4, 5, 6}};
for (int row = 0; row < jagged.length; row++) {
    for (int col = 0; col < jagged[row].length; col++) {
        System.out.print(jagged[row][col] + " ");
    }
    System.out.println();
}
```

We will return to two-dimensional arrays later with more detailed memory diagrams and algorithms.

14 Worked Example: Analyze a Class's Scores

This complete example combines indexing, traversal, averages, counting, and a method. Notice that it checks for an empty array before calculating an average.

```
import java.util.Arrays;
public class ScoreReport {
    public static double average(int[] scores) {
        if (scores.length == 0) {
            throw new IllegalArgumentException("No scores");
        }
```

```
        int sum = 0;
        for (int score : scores) {
            sum += score;
        }
        return (double) sum / scores.length;
    }

    public static void main(String[] args) {
        int[] scores = {75, 90, 82, 96, 90};
        int atLeast90 = 0;
        for (int i = 0; i < scores.length; i++) {
            if (scores[i] >= 90) {
                atLeast90++;
            }
        }
        System.out.println(Arrays.toString(scores));
        System.out.println("Average: " + average(scores));
        System.out.println("At least 90: " + atLeast90);
    }
}
```

The program prints `[75, 90, 82, 96, 90]`, then `Average: 86.6`, then `At least 90: 3`. The average is 433 divided by 5. We used `scores.length` so the same code works for any nonempty array length.

15 Final Comparison of Common Array Operations

Expression	Meaning
`int[] a = new int[5]`	Create five integer elements, initially zero
`a[0]`	First element
`a[a.length - 1]`	Last element, if nonempty
`a.length`	Number of elements (field)
`a == b`	Same array object?
`Arrays.equals(a,b)`	Equal element values?
`Arrays.toString(a)`	Readable one-dimensional array display
`Arrays.copyOf(a,n)`	New array with requested length
`Arrays.copyOfRange(a,1,4)`	Copy indices 1, 2, 3
`a.clone()`	New shallow-copy array
`a[1:4]`	Invalid Java syntax (Python-style slicing)